四项目评测标准 v1.0（可复跑）

这份文档是 2026-11-02 两次实测（全功能对比 / 小白易用性）沉淀下来的同一把尺。
目的：以后换项目、换版本、加第五家，照着这份跑一遍就能得到可比的分，不靠印象。

- 立标时间：2026-11-02
- 统一模型：deepseek-ai/DeepSeek-V4.1-Flash
- 产出目录：.tmp/bench/（证据） + 桌面（报告）
- 首次基线：见第 10 节

---

1. 两条口径，别混用

口径	问的问题	维度数	满分	打分锚点在哪	用途
A · 全功能评测	它能做到什么，做得多深	14	140	§3.2	选型、技术评审、竞品分析
B · 小白易用性	一个不写代码的人能不能上手	10	100	§4.5	推荐给非技术用户、决定"先装哪个"

两套口径共用：独有能力矩阵（§3.3）、Token 规程（§5）、仿真测试规程（§6）、产出与命名（§8）、复跑 SOP（§9）。

铁律：两条口径必须分开跑、分开出报告。不要合成一个总分——"功能强但装不上"和"好装但没灵魂"是两种完全不同的结论，混在一起就废了。

---

2. 通用纪律（两次实测证明这几条必须写死）

1. 只读被审项目。审计前后各跑一次 git status，报告里写清楚"被审仓库干净"。
2. 一个项目一个审计员，四家并行；审计员之间互相看不到对方分数，避免抄分、避免对齐全场。
3. 不许编数字。跑不起来就写 not_run / blocked: <真实报错首行>，这一项按静态证据给分。
4. 每个分数必须有证据：文件:行号 或 命令输出原文。没有证据的判断直接删掉。
5. 对自家项目同样苛刻。自家项目如果有"少一步就废"的坑，必须写进 blockers。
6. 功能缺失要标 missing，不要用"正在做/规划中"糊过去。

---

3. 口径 A：全功能评测（14 维 × 0–10 = 140）

3.1 维度定义

#	key	维度	看什么
1	deploy	部署与运行门槛	依赖、一键启动、Docker、Windows 原生支持
2	platforms	消息平台覆盖	QQ/微信/TG/Discord/飞书/Slack/Web…按适配器个数与成熟度
3	persona	人格与角色扮演	人设结构、长期一致性、多角色切换
4	memory	记忆系统	短/长期、按人分区、召回、遗忘曲线、可编辑
5	relation	关系与情感模型	好感/信任/厌恶等数值、阶段、恋人/家人/否决等社会规则

#	key	维度	看什么
6	proactive	主动性与生活感	主动外发、日程、念头、身体状态、夜间静默
7	social	社交执行力	主动联系第三方、点歌翻唱、语音条、表情包、群管理动作
8	voice	语音与多模态	TTS 音色克隆、ASR、图像理解、视频/音乐处理
9	tools	工具与插件生态	插件市场、MCP、工具环、权限
10	webui	WebUI 完整度	面板、可视化、可操作性（不是"有个页面"就算）
11	engineering	工程质量	测试数量与可运行性、CI、lint、类型、模块边界
12	privacy	数据与隐私	本地优先、无云依赖、数据隔离、导出/迁移
13	docs	文档与可维护性	README、架构文档、贡献指南、更新日志
14	perf	性能与资源占用	冷启动、单轮延迟、内存/显存、并发

3.2 打分锚点（每一维都用这套，别自由发挥）

分	含义
10	能直接上生产，有测试兜底，有文档，边界情况想过
7–9	完整可用，少数缺口（缺测试 / 缺某种场景 / 平台少一个）
4–6	有主体实现，边角要自己补，或完全没有测试
1–3	只有雏形；或只有硬编码版本；或必须大量自己写
0	完全没有（此时 status 必须是 missing）

3.3 独有能力矩阵（比分数更重要）

某一家有、另一家没有的能力，缺失方对应位置写 未实现，并登记：

```
{"feature": "按人分区记忆 + 关系四维 + 冷归档还原", "owner": "SMD-P",
 "others": ["qq-bridge", "surfcore", "AstrBot"]}
```

规矩：unique
只登记"结构性差异"，不登记"参数多少"这种程度差异；每条必须能指到对方仓库里"搜不到"的事实（例如全仓 grep 命中 0）。

---

4. 口径 B：小白易用性（10 维 × 0–10 = 100）

4.1 只走最省事的那条路

项目同时提供 Docker / 安装包 / 一键脚本 / 源码安装时，只走最省事那条；另一条最多一句"还有更麻烦的路"。

4.2 维度定义

#	key	维度	看什么
1	install	安装方式	有没有安装包 / 一键脚本；要不要先自己装运行时
2	deps	前置依赖	Python/Node/Docker/数据库/编译工具/另一个软件，越少越好
3	steps	步骤数量	从"下载完"到"发出第一条消息"要几步（≤3 步 = 10）

#	key	维度	看什么
4	gui	图形界面	能不能不碰命令行、不碰配置文件
5	config	配置难度	表单填写 / 改 YAML / 改 JSON / 改代码
6	defaults	默认可用	不填 key、不改配置，能不能先跑起来（能不能看到界面）
7	diagnose	出错可诊断	报错能不能看懂、有没有自检、日志在哪
8	doc	文档友好	中文、有截图、有"下一步点哪"
9	recover	恢复与升级	备份、回滚、重装、清数据
10	total_time	首次跑通耗时	≤10 分钟 = 10；>2 小时 = 1

4.3 步骤计数规则

- 一次需要人动手的决策/点击/输入 = 1 步；纯等待不算。
- 每步标 need_cli（要不要敲命令）和 need_doc（不看文档会不会）。
- "装别人的软件"也各算 1 步（装 Node、装 DSH、装 NapCat、扫码登录…都算）。
- 只统计到"发出第一条消息"。想接 QQ 群聊的额外步骤单独写进 steps_summary，不混进主步骤数。

4.4 卡点（blockers）必须写真名

每条至少给：where（第几步）、what（技术上怎么回事）、confusion（人会在哪儿懵）。

"静默失败"要特别标出来（界面正常、脚本不报错、但实际不通）——这类是小白杀手。

推荐的 JSON 写法：{"where": "...", "what": "...", "confusion": "...", "silent": true}。

4.5 打分锚点（口径 B 专用，每一维都用这套）

这一节是口径 B 的核心。缺了它，四个人打出来的分不可比——10 分必须是同一种"省事"。

install 安装方式

分	长什么样
10	官方安装包（.exe/.msi/.dmg），双击下一步到底
7	有官方一键脚本，但要敲一行命令
4	只能源码装：自己装运行时 + npm install / pip install
1	连安装说明都要靠翻 issue 才能拼出来；或安装包不在本仓库、要别人发你

deps 前置依赖

分	长什么样
10	装完那个包，什么都不用再装
7	再装一个"人人都有"的东西（浏览器级别）
4	要自己装运行时（Python/Node）+ 一个 API key
1	要装 3 个以上外部软件（运行时 + 宿主程序 + QQ 框架），且版本互相咬

steps 步骤数量

分	长什么样
10	≤3 步
7	4–6 步
4	7–10 步
1	≥11 步，且中途要查文档才知道下一步

gui 图形界面

分	长什么样
10	全程点鼠标：安装向导 + 面板 + 表单配置，黑窗口一次都不出现
7	有面板，但首启要敲一次命令
4	有面板，配置仍要手改文本文件
1	全程命令行；面板是"另一个项目的"或压根没有

config 配置难度

分	长什么样
10	面板里填表：选模板 + 粘贴 key + 保存
7	面板里填表，但有几项必须去别处抄（端口/token/白名单）
4	手改 JSON/YAML，字段名看不懂，改错不报错
1	要改代码，或要理解内部数据流才能填对

defaults 默认可用

分	长什么样
10	装完直接能对话（自带可用模型/离线兜底）
7	不填 key 也能看到界面、点得动，会明确提示"去填 key"
4	不填 key 界面正常但完全无响应且不报错（静默）
1	不填 key 连启动都不行，或直接退出

diagnose 出错可诊断

分	长什么样
10	报错中文、指到具体文件行；有自检/诊断页一键体检
7	报错能看懂，日志有固定路径，FAQ 覆盖常见坑
4	报错是英文堆栈或空对象 错误：{}，日志位置要自己找
1	失败完全静默（窗口一直刷同一行 / 界面正常但没反应）

doc 文档友好

分	长什么样
10	中文、有截图、按步骤写"点哪个/填什么"，含常见问题
7	中文文档完整，但没有截图或少量过时
4	只有英文/只有 README，快速开始默认你已装好前置
1	文档与代码不符（教的端口/版本是错的），或要读代码才知道怎么跑

recover 恢复与升级

分	长什么样
10	界面里能备份/恢复/升级；一键清数据；重装不留垃圾
7	有脚本或文档写清备份路径与升级方式
4	升级要自己 git pull / 覆盖文件；数据要自己找
1	升级即毁配置；或不知道数据存在哪

total_time 首次跑通耗时

分	长什么样
10	≤10 分钟
7	10–30 分钟
4	30–90 分钟
1	>2 小时，或当天跑不通

一致性校验：打完后自检两件事——

① steps 与 total_time 不能矛盾（12 步却写 5 分钟 = 错）；

② defaults=10 的项目不该在 blockers 里出现"没填 key 没反应"。

4.6 first_run_minutes 怎么给

- 定义：从"下载完"到"发出第一条消息" 的时间，按"完全不会电脑的人"估，不按开发者估。
- 必须给区间 first_run_minutes_range: [下限, 上限]，并写 first_run_basis（依据：文档步骤数 / 你实测的某一段）。
- 不许照抄项目自报的"20 分钟"——作者的时间是"前置全装好"的口径，要换算成小白口径并说明换算理由。
- 拆分建议：装运行时 X 分钟 + 装宿体 Y 分钟 + 注册/扫码 Z 分钟 + 配置 W 分钟，加总后再给区间。

4.7 口径 B 的证据要求（比 A 更硬）

要什么	怎么算合格
oneclick	必须指到真实存在的文件（install.bat:1、docker-compose.yml:1）。搜不到就写 [] 并在 oneclick_gaps 说明"文档说有但仓库里没有"
prereqs	全部前置都要列，包括"另一个软件的某个版本"；漏一个 = 这份审计不合格
steps	每步都要有 need_cli 与 need_doc，不许留空
blockers	≥2 条真实卡点，带 where；静默失败必须单独标出
scores	每维都要 evidence，指到文档行号或脚本行号
tokens	口径 B 也要 token（同五场景 + 组件拆解），别只做 A

4.8 口径 B 的三个常见误判（本次实测都踩到过）

1 把"文档写得好"当"能跑通"：README 里 6 步，前提是宿主软件已装好、QQ 框架已登录。要按"从零"重数。

2 把"有 GUI"当"不用配置"：面板里仍要手填端口 token 白名单，那一栏不认识照样卡死，config 该扣分。

3 把"作者 20 分钟"当"小白 20 分钟"：作者手里有编译好的环境、知道日志在哪。小白连"日志在哪"都是第 7 步。

---

5. Token 计量规程（两口径都要）

5.1 两个数一起记，缺一不可

数	怎么来	意义
counted_cl100k	tiktoken 的 cl100k_base 离线数	可复现、零成本，但对中文偏差极大
provider_prompt_tokens	把同一段文本真发给 v4.1-flash，读 usage.prompt_tokens	精确值，就是真实计费口径

实测偏差范围：-14% ~ +73%（同一份中文 prompt，cl100k 可能高估 73%）。所以不能只报 cl100k。

5.2 五个固定场景（不许增删）

场景	输入
chat	「今天有点累，不太想说话。」
group	群聊三人，最后一句 @她「你昨天说的那个游戏叫啥」
recall	「我上次跟你说我妈住院的事，你还记得吗」
tool	「帮我查一下今天的汇率」
love	「我喜欢你」

5.3 三条硬规矩

1 用来计量的 prompt 必须落盘：.tmp/bench/prompts/<project>_<scenario>.txt，别人能复算；不许只贴一个数字。

2 同一段只 provider 调一次（工具有哈希缓存，重复调会命中 cached: true）。

3 三口径对账：cl100k / provider / 项目自己 API 返回的 tokens_in —— 三个数都要写；

provider 与 API tokens_in 偏差应 < 1%，超了说明 prompt 抓错了（实测 SMD-P 偏差 0.16%–0.52%）。

5.4 组件拆解（每个项目一份）

把一次 chat 的 prompt 按构建来源拆块，每块给 token 数与 file:line：

系统常驻 / 人格卡 / 记忆注入 / 工具 schema / 历史对话 / 本轮用户消息。

拆块之和应约等于整份（差值是分隔符/空行，要说明）。

5.5 计量工具

```
# 工作目录：C:\AIGCProgram\SMD\SMD-P  解释器：.\.venv\Scripts\python.exe
python .tmp/bench/tok.py compare <prompt文本文件>  # 两个数 + 偏差
python .tmp/bench/tok.py count "<一段文本>"       # 只数 cl100k
python .tmp/bench/tok.py provider <文件>          # 只调模型读 usage
```

---

6. 仿真测试规程（口径 A 的硬证据）

1. 先找它自己声明的测试命令（package.json scripts / Makefile / pyproject / pytest.ini / scripts/test-*.mjs）。
2. 按顺序尝试，记录：cmd / exit / passed / failed / note。
3. 依赖缺失导致的失败 ≠ 项目失败，写 blocked: <真实报错首行>。
4. 附加要跑的：lint（ruff check / eslint）、类型（tsc）、CI 配置是否存在（.github/workflows）。
5. 结论里要区分三种"绿"：
6. 真绿：全量跑通；
7. 降级绿：mock/兜底才过（例如 SMD-P 的 smoke 日志里仍有 429 限流，是靠降级兜底绿）——必须写明；
8. 假绿：只跑了子集或跳过一堆。

---

7. 评分合成

- 口径 A：14 维直接相加（满分 140）；并列时看 relation + memory 谁高（这两项最难伪装）。
- 口径 B：10 维直接相加（满分 100）；并列时看 steps 谁少。
- 必须同时给"能力分"和"易用分"，并在报告里一句话说清二者冲突：

"最容易装的，不一定是最后最适合你的。"

- 不允许为了好看做加权调和；权重留给读者自己做。

---

8. 产出物与命名

```
.tmp/bench/
  SPEC.md              口径 A 标准（本项目历史版本）
  SPEC2.md             口径 B 标准
  <project>.json       口径 A 原始审计（14 维 + runtime + unique + tokens）
  easy_<project>.json  口径 B 原始审计（10 维 + steps + blockers + tokens）
  prompts/<project>_<scenario>.txt   计量用 prompt 原文（可复算）
  tokcache.json        provider 计量缓存（哈希 → usage）
  tok.py               计量工具
  report.py            口径 A → PDF
  report2.py           口径 B → PDF
  基线-2026-11-02.md   本次基线结论（本文第 10 节）
桌面/
  四项目对比报告-<date>.pdf        口径 A
```

9. 快速复跑 SOP（照着做就行）

1 准备：把被审项目 clone 到 C:\AIGCProgram_compare\<name>（git clone --depth 1）。
2 改标准（只在换维度/换场景时）：改 SPEC.md / SPEC2.md，别改了不记。
3 派审计：每个项目一个子代理，context=fresh，只读被审项目，只写自己的

<project>.json 与 easy_<project>.json；提示词里贴 SPEC 全文 + 明确"不编数字"。

1 等回报：四个都回来后，用 python .tmp/bench/summary.py 与 easy_summary.py 验证 JSON 可解析、维数齐、总分可在 30 秒内复算。
2 抽查复算：随机抽 2 份 prompt 重新 tok.py compare，与子代理报的数必须一致

（cached: true 且哈希相同 = 同一段文本，可信）。

1 出报告：

```
python .tmp/bench/report.py  <口径A输出.pdf>
python .tmp/bench/report2.py <口径B输出.pdf>
```

1 归档：把 PDF 拷到桌面，日期进文件名；.tmp/bench/ 里的 JSON 与 prompts 不要删，

下次做版本漂移对比要用。

1 记录漂移：新结果与第 10 节基线逐维对比，变化 >1 分的维度必须解释原因（版本变更 / 标准调整 / 审计员口径差异）。

10. 基线（2026-11-02 首测，作为后续对比锚点）

口径 A • 全功能（满分 140）

项目	总分	最强的两项	最弱的两项
AstrBot	99	平台矩阵 10、工具生态 9	记忆 3（missing）、关系 0（missing）
surfcore	94	文档 9、WebUI 8	关系 3、性能 5
SMD-P	90	记忆 9、关系 9	平台 3、工具 3
qq-bridge	85	主动 9、社交 8	关系 3、记忆 5

口径 B • 小白易用（满分 100）

项目	总分	步骤	敲命令	首次跑通
AstrBot	73	7	1 步	≈20 分钟
surfcore	47	12	0（有 kit）	≈90 分钟
SMD-P	42	12	4 步	≈180 分钟
qq-bridge	38	13	6 步	≈75 分钟

Token（v4.1-flash，五场景均值）

项目	prompt tokens/轮
AstrBot	299
surfcore	2,575
SMD-P	3,279

项目	prompt tokens/轮
qq-bridge	7,164

11. 已知坑（踩过一次，别再踩）

1. sqlite3.Row 没有 .get() —— db.one() 返回 Row，(row or {}).get(...) 直接 AttributeError。先 dict(row)。
2. CRLF/LF 混写会毁掉文件：PowerShell 的 -replace 写回时用

[System.IO.File]::WriteAllText(path, text, (New-Object System.Text.UTF8Encoding($false)))，别用 Set-Content。

1. reportlab 的 Paragraph 是 XML：所有动态文本必须 esc()，否则 < 会当标签、& 会炸。
2. 表格里别放裸字符串：Table([["中文"]]) 会用 Helvetica → 方块。中文一律 Paragraph(..., style)。
3. 端口是静默杀手：SMD-P 上报必须 8787（/onebot/event 无 /v1 前缀），写成 8788 就零报错零消息。
4. cl100k 对中文高估严重（本次最高 +73%），别拿它估成本。
5. mock 环境会掩盖真实行为：mock 下"没 key"不报错、min_days 这类真实门槛也会被跳过；功能结论要在真 key 下验。
6. 没有 CI 的项目，别把"测试通过"当默认：先找 npm test 脚本是否存在（qq-bridge 就是 Missing script: "test"）。

12. 附录：JSON Schema

12.1 口径 A（<project>.json）

```
{
  "project": "astrbot",
  "commit": "<short hash>",
  "loc": {"python": 229152, "js": 98514, "total_files": 1926},
  "runtime": [{"cmd": "pytest -q", "exit": 0, "passed": 5275, "failed": 6, "note": "..."}],
  "scores": {
    "memory": {"score": 3, "evidence": "db/po.py 20 张表无记忆列", "status": "missing"}
  },
  "unique": [{"feature": "...", "owner": "surfcore", "others": ["SMD-P", "qq-bridge", "astrbot"]}],
  "tokens": {
    "model": "deepseek-ai/DeepSeek-V4.1-Flash",
    "scenarios": [{"name": "chat", "prompt_file": "prompts/x_chat.txt", "chars": 964,
                   "counted_cl100k": 220, "provider_prompt_tokens": 256,
                   "delta_pct": -14.1, "max_out": 1024, "note": "真实运行"}],
    "summary": {"avg_prompt_tokens": 299, "method": "provider 实测"}
  },
  "notes": "定位 + 最强两点 + 最弱两点"
}
```

12.2 口径 B（easy_<project>.json）

```
{
  "project": "surfcore",
  "easiest_path": "别人发 install-kit zip → 双击 2-安装.bat → 重启 DSH → GUI 一键安装 → @她",
  "steps": [{"n": 1, "action": "装 Node ≥22.13", "how": "官网 msi 双击", "need_cli": false, "need_doc": true}],
  "prereqs": ["Node 22.13+", "DSH 0.2.1-rc.2+", "SnowLuma2", "API key"],
  "oneclick": ["2-安装.bat", "3-自检.bat"],
  "blockers": [{"where": "第 6 步", "what": "白名单默认空 = 谁都不理",
                "confusion": "装完 @ 没反应，会删掉重装"}],
  "scores": {"steps": {"score": 3, "evidence": "12 步，见 steps 表"}},
  "first_run_minutes": 90,
  "first_run_minutes_range": [45, 120],
  "tokens": {"model": "...", "scenarios": [], "breakdown": [{"part": "常驻指令", "tokens": 3306, "from": "bridge.
js:12603"}]},
```

```
  "verdict": "白话结论：能装上、也能用，但..."
}
```

---

13. 一句话记住

口径 A 回答"它有多强"，口径 B 回答"你能不能上手"。两个都测，两份报告，别合并。